

Supplementary Material: CL-TAV Contracts, Wrappers and Decision Tables

Author Name (affiliation pending)

Abstract

This supplement is part of the delivered manuscript package. It typesets ALG-CLTAV-02/03/04 wrappers, ALG-CLTAV-05/06/07 cores, typed records and the A/P stop tables. It does not change ALG-CLTAV-01 and does not report experiments. Supplement algorithm numbers are S1, S2, ...; they are not the main-text Algorithm 1.

1 Scope and numbering

Main-text Algorithm 1 is ALG-CLTAV-01. Supplement Algorithms S1–S6 are ALG-CLTAV-02, 03, 04, 05, 06 and 07. S9 in the main text calls COMMITCOMPATIBLEUPDATE (S6 / ALG-CLTAV-07), not a bare IF-HIST-UPDATE invocation.

2 S0–S10 duty map

Step	Call	Duty / exception
S0	INITIALIZE	Γ, η, H, U ; distinct k , charges, retry
S1	IF-RESOURCE-STOP with IF-EQUIV	already decided Admit A2–A5 or P-stop
S2	PREDICTCURRENT	ALG-CLTAV-02; TEST-scoped GAP
S3	SELECTANDADMIT	ALG-CLTAV-05; freeze ξ including U
S4	CHARGEONCE	one charge; retry cap $\neq$ budget
S5	IF-EXECUTE-RECORD	raw record and effect class
S6	INTERPRETOUTCOME	ALG-CLTAV-03 + timing ALG-CLTAV-06
S7–S8	RESOLVEOUTCOME	ALG-CLTAV-04; adopt returned contexts
S9	COMMITCOMPATIBLEUPDATE	version/intersection; SPEC-ERROR; summary commit
S10	IF-RESOURCE-STOP with IF-EQUIV	P-stop or continue to S1

3 Typed records

Record	Fields
PredictionResult	status OK GAP; reason; classesByTest (OK); historyVersionUsed; uncertaintyRef
Decision	kind ACTION EXIT SPEC-ERROR; actionKind; actionId; classesUsed; reason
SelectSnapshot ξ	actionId; actionKind; qUsedAtSelect; historyVersion; classesUsedAtSelect; uncertaintyRef
ExecutionResult	attemptId; record; effect; correlationId
Outcome z	validity; effect; I_z or NONE; summaryConfirmed; postSummary; timing
Resolution δ	control COMMIT RETRY STOP; Γ' ; η' ; outcome; reason

4 Admit and post-update stops

A2 no affordable distinguishing TEST and no eligible Prep/Recover. A3 named prediction-gap when Recover/Prep are ineligible. A4 retry-cap is not used here; retry-cap is Stop-Error. A5 Recover eligible but unaffordable (Stop-Budget). P1 Stop-Empty; P2 Stop-Singleton; P3 Stop-645; P4 Stop-Equivalent; P5 Stop-Budget after update. Stop-NoDistinguisher is exclusive with those P-stops when no remaining distinguisher exists and no P-stop already fired. Order is Empty, Singleton, 645, Equivalent, Budget. Singleton h_{normal} is not protocol PASS.

5 Short arguments

Monotonicity, retention (R1)–(R5), one-step minimax and resource termination are stated in the main text. The algebraic details are the accepted method §3.9.1 arguments: update is intersection; **ERROR** does not exclude; minimax is a least element of a finite selectable set; billed executions are bounded by the selected mode. They are not global optimality and not independent mathematical approval.

6 AI disclosure fields

System: PENDING-USER (AUTH-AI). Scope (text/figures/code): PENDING-USER. Author verification: required; not completed by this sentence. IEEE AESS guidance is rechecked at submission.

7 Wrappers and core mechanisms

Algorithm S1: PREDICTCURRENT: current-observation prediction wrapper (ALG-CLTAV-02, S2)

Require: SessionContext Γ ; HistoryHandle η ; current H ; uncertainty reference U ; action library L
Ensure: PredictionResult: status OK with nonempty classesByTest, or status GAP with a reason; the session's own η is read, not rebuilt

```
1  $raw \leftarrow \text{IF-PRED-OBS}(\Gamma, \eta, H, U, L);$   
  // branch on the backend tag BEFORE reading or projecting classes; a GAP result carries no classesByTest  
2 if  $raw.status = \text{GAP}$  then  
3   | return PredictionResult(status = GAP, reason =  $raw.reason$ );  
  // the gap stays TEST-scoped  
4  $classes \leftarrow \text{PROJECT-ONTO-CURRENT-H}(raw.classes, H);$   
  // a declared class set that projects to no nonempty class is the same named gap, not OK and not  $score = 0$   
5 if  $classes$  has no nonempty member then  
6   | return PredictionResult(status = GAP, reason = no currently valid class);  
7 return PredictionResult(status = OK, classesByTest =  $classes$ , historyVersionUsed =  $\eta.version$ , uncertaintyRef =  $U$ );
```

Algorithm S2: INTERPRETOUTCOME: execution and observation coordination (ALG-CLTAV-03, S6)

Require: ExecutionResult er ; SelectSnapshot ξ ; Γ ; η ; uncertainty reference U
Ensure: a total Outcome z : the execution effect, the interpretation effect class and the evaluated confirmation are combined by one decision table; skipped confirmation is NOT-EVALUATED, not an evaluated false

```

1 // total constructor; the executed effect and the snapshot kind are copied, never left to an implicit global
2  $z \leftarrow \text{NewOutcome}(); z.\text{effect} \leftarrow er.\text{effect}; z.\text{kind} \leftarrow \xi.\text{actionKind}; z.\text{actionId} \leftarrow \xi.\text{actionId};$ 
3 // CONFIRMED-NOT-SENT and UNKNOWN-EFFECT do not call an observation or confirmation interface that did not run
4 if  $er.\text{effect} \in \{\text{CONFIRMED-NOT-SENT}, \text{UNKNOWN-EFFECT}\}$  then
5   | return  $z$ ;
6   |  $(I_z, \text{effectClass}, \text{summaryConfirmed}, \text{postSummary}, \text{ownership}, \text{measurementInterval}) \leftarrow$ 
7     |  $\text{IF-OBS-INTERPRET}(\Gamma, er.\text{record}, \text{clocks}, \varepsilon);$ 
8     | // an invalid measurement or an interpretation ERROR is propagated to S7 and never confirmed
9   if  $\text{ownership}$  is invalid then
10    |  $z.\text{validity} \leftarrow \text{ERROR}; z.\text{effect} \leftarrow \text{UNKNOWN-EFFECT}; \text{return } z;$ 
11    |  $\text{timing} \leftarrow \text{INTERPRETTIMEDOBSERVATION}(\text{ownership}, \text{measurementInterval}, \xi, N_r, D_r, \text{Match}_r);$ 
12    | if  $\text{timing.verdict} = \text{ERROR}$  then
13      |  $z.\text{validity} \leftarrow \text{ERROR}; z.\text{effect} \leftarrow \text{UNKNOWN-EFFECT}; \text{return } z;$ 
14    |  $z.\text{validity} \leftarrow \text{timing.verdict}; z.I_z \leftarrow I_z;$ 
15    | // Prep/Recover confirmation runs BEFORE the effect is finalized, so a confirmed actual successor is not overwritten
16    | by an earlier UNKNOWN
17  if  $\xi.\text{actionKind} \in \{\text{PREP}, \text{RECOVER}\}$  then
18    |  $z.\text{prepResultEvaluated} \leftarrow \text{true};$ 
19    |  $(z.\text{targetConfirmed}, \text{summaryConfirmed}, z.\text{prepErr}, z.\text{declaredTarget}, z.\text{evidence}, \text{postSummary}) \leftarrow$ 
20      |  $\text{IF-PREP-RECOVER}(\Gamma, \eta, \xi.\text{actionId}, er.\text{record});$ 
21      | // IF-PREP-RECOVER returns values only; it does not write  $\Gamma$  or  $\eta$ 
22   $z.\text{summaryConfirmed} \leftarrow \text{summaryConfirmed}; z.\text{postSummary} \leftarrow \text{postSummary};$ 
23 // one effect decision table: propagate the interpretation effect class; an unconfirmed successor or target becomes
24 UNKNOWN-EFFECT
25 if  $\text{effectClass} = \text{UNKNOWN-EFFECT}$  or  $z.\text{prepErr}$  or  $(\xi.\text{actionKind}$  is Prep and  $z.\text{prepResultEvaluated}$  and not
26  $\text{summaryConfirmed})$  or  $(\xi.\text{actionKind}$  is Recover and  $z.\text{prepResultEvaluated}$  and not  $z.\text{targetConfirmed})$  or
27  $(\text{effectClass} = \text{NONE}$  and not  $\text{summaryConfirmed})$  then
28   |  $z.\text{effect} \leftarrow \text{UNKNOWN-EFFECT};$ 
29 else
30   |  $z.\text{effect} \leftarrow \text{effectClass};$ 
31   | // a confirmed normal TEST/Prep/Recover keeps the interpretation effect class
32 return  $z$ ;
33 // values only; all  $\Gamma/\eta$  writes stay in ALG-CLTAV-04 and S9

```

Algorithm S3: RESOLVEOUTCOME: ERROR, retry and confirmed recovery (ALG-CLTAV-04, S7-S8)

Require: Outcome z (effect, kind, validity, confirmation); SelectSnapshot ξ ; Γ ; η
Ensure: Resolution $\delta = (\text{control}, \Gamma', \eta', \text{outcome}, \text{reason})$: Γ' initialized from Γ

```

1  $\Gamma' \leftarrow \Gamma; \eta' \leftarrow \eta;$ 
2 // copy/persist semantics: the returned contexts are total, never undefined
3 if  $z.\text{validity} = \text{ERROR}$  or  $z.\text{effect} \in \{\text{CONFIRMED-NOT-SENT}, \text{UNKNOWN-EFFECT}\}$  or  $z.\text{prepErr}$  or  $(z.\text{kind}$  is Prep and
4  $z.\text{prepResultEvaluated}$  and  $z.\text{summaryConfirmed}$  is false) or  $(z.\text{kind}$  is Recover and  $z.\text{prepResultEvaluated}$  and
5  $z.\text{targetConfirmed}$  is false) then
6   | // S7 classify by kind and evaluated confirmation; do not fold CONFIRMED-NOT-SENT into UNKNOWN
7   | do not exclude  $H$ ;
8   | if  $z.\text{validity} = \text{ERROR}$  or  $z.\text{prepErr}$  or  $z.\text{effect} = \text{UNKNOWN-EFFECT}$  or  $(z.\text{kind}$  is Prep and  $z.\text{prepResultEvaluated}$  and
9     |  $z.\text{summaryConfirmed}$  is false) then
10     | mark  $\eta'$  conservative-unknown;  $\Gamma'.\text{qStatus} \leftarrow \text{UNKNOWN}$ ; clear  $\Gamma'.\text{currentSummary}$ ;
11   else if  $z.\text{effect} = \text{CONFIRMED-NOT-SENT}$  then
12     | leave  $\Gamma'.\text{qStatus}$  and compatible history unchanged;
13   else
14     |  $\Gamma'.\text{qStatus}$  stays UNKNOWN; clear  $\Gamma'.\text{currentSummary}$ ; mark  $\eta'$  conservative-unknown;
15     | // unconfirmed Recover target
16    $\Gamma'.\text{retryCount} \leftarrow \Gamma.\text{retryCount} + 1;$ 
17   if  $\Gamma'.\text{retryCount} \geq \Gamma'.\text{retryCap}$  then return Resolution( $\text{control} = \text{STOP}, \Gamma', \eta', z, \text{reason} = \text{Stop-Error}$ );
18   return Resolution( $\text{control} = \text{RETRY}, \Gamma', \eta', z$ );
19   // an ERROR/unknown outcome reaches retry, never commit
20 else if  $z.\text{kind}$  is Recover then
21   | // S8 a confirmed Recover is a necessary condition for this branch; sole writer of KNOWN; the normalized summary
22   | returns to S9
23   |  $\Gamma'.\text{qStatus} \leftarrow \text{KNOWN}; z.\text{postSummary} \leftarrow (z.\text{declaredTarget}, z.\text{evidence});$ 
24 return Resolution( $\text{control} = \text{COMMIT}, \Gamma', \eta', z$ );
25 // a normal TEST/Prep with a valid observation commits; the normalized Outcome carries the confirmed postSummary to
26 S9

```

Algorithm S4: SELECTANDADMIT: test selection and resource admission (ALG-CLTAV-05, S3)

```

Require:  $\Gamma; \eta$ ; current  $H$ ; action library  $L$ ; typed  $pred$  (PredictionResult)
Ensure: a Decision: kind ACTION / EXIT / SPEC-ERROR with actionKind, actionId, classesUsed and reason
// state eligibility is independent of affordability; under UNKNOWN only an eligible Recover qualifies, never a Prep
1 eligible( $t$ )  $\leftarrow$  ( $\Gamma.qStatus = UNKNOWN \ ? \ (t.kind = RECOVER \wedge t.eligibleUnknown) : (t.kind \neq RECOVER \wedge t.enabledAt = \Gamma.currentSummary)$ );
2  $A \leftarrow \{t \in L : eligible(t)\}$ ;
// affordability is decided by IF-SELECT-ADMIT, not assumed from eligibility
// a global or per-TEST projection gap withholds TEST scoring and keeps the gap identity; it does not itself block eligible Prep/Recover
3  $testGap \leftarrow false$ ;
4 if  $pred.status = GAP$  then
5    $C \leftarrow \{\}$ ;  $testGap \leftarrow true$ ;
6 else
7    $C \leftarrow \{C_t\}$  with  $C_t \leftarrow \{H \cap C : C \in pred.classesByTest[t], H \cap C \neq \emptyset\}$  for each TEST  $t \in A$ ;
8   if a TEST  $t \in A$  has declared classes but  $C_t = \emptyset$  then
9      $C \leftarrow \{\}$ ;
10     $testGap \leftarrow true$ ;
// IF-SELECT-ADMIT is the single admission authority: affordability and A1-A5; exclusive kind; one-step minimax
 $s(t) = \max_{C \in C_t} |C|$  over currently valid nonempty classes of affordable distinguishing TESTs  $D \cap F$  only; cost/id tie-break; never argmin  $\emptyset$ ; Prep/Recover are not observation minimax; uninformative TESTs are not A2
11 ( $k, S, t^*, admit$ )  $\leftarrow$  IF-SELECT-ADMIT( $A, \Gamma.currentSummary, C, \Gamma, \eta, H$ );
12 if  $k = ADMIT$  then
13   if  $testGap$  and  $admit = A3$  then return Decision(kind = SPEC-ERROR, reason =  $pred.reason$  or per-test prediction gap);
14   return Decision(kind = EXIT, reason = the A2-A5 admission reason);
15 else if  $k = PREP$  then
16   return Decision(kind = ACTION, actionKind = PREP, actionId =  $t^*$ , classesUsed =  $S$ );
17 else if  $k = RECOVER$  then
18   return Decision(kind = ACTION, actionKind = RECOVER, actionId =  $t^*$ ,  $\cdot$ );
19 else if  $testGap$  then
20   return Decision(kind = SPEC-ERROR, reason =  $pred.reason$ );
21 return Decision(kind = ACTION, actionKind = TEST, actionId =  $t^*$ , classesUsed =  $C_{t^*}$ );

```

Algorithm S5: INTERPRETTIMEDOBSERVATION: timed observation kernel (ALG-CLTAV-06, S6 kernel)

```

Require: the single ownership result and the raw measurement interval from IF-OBS-INTERPRET; SelectSnapshot  $\xi$ ; named obligation inputs  $N_r$  (allowed interval) and  $D_r$  (physical domain); declared match policy  $Match_r$ ; declared uncertainty  $\varepsilon$ 
Ensure: a TimingResult: verdict in {PASS, FAIL, INCONCLUSIVE, ERROR}; the single pairing evidence; the interval used; validity
// resolve  $N_r, D_r$  and  $\varepsilon$  from the named obligation and instance inputs, not from free symbols
1 pair  $\leftarrow Match_r(ownership, \xi)$ ;
// one pairing result feeds T2 and the no-response rule
2 if pair is ambiguous then return TimingResult(verdict = ERROR);
3 if pair is a legitimate trigger/response pair then
4    $M \leftarrow measurementInterval \cap D_r$ ;
5   if  $M = \emptyset$  then return TimingResult(verdict = ERROR);
6   if  $M \subseteq N_r$  then
7     return TimingResult(verdict = PASS, evidence = pair,  $M$ );
8   else if  $M \cap N_r = \emptyset$  then
9     return TimingResult(verdict = FAIL, evidence = pair,  $M$ );
10  return TimingResult(verdict = INCONCLUSIVE, evidence = pair,  $M$ );
// open/closed endpoints preserved
// a missing response forms a no-response only after a still-active instance reaches its deadline; this round not seeing it is not a timeout
11 if the obligation is still active and expired then
12   // use the earliest admissible elapsed horizon (a scalar), with the requirement's boundary convention
13   if  $N_r$  is closed at  $U_r$  then
14     expired  $\leftarrow lowerHorizon > U_r$ ;
15   else
16     expired  $\leftarrow lowerHorizon \geq U_r$ ;
17   if expired then return TimingResult(verdict = FAIL);
18   return TimingResult(verdict = INCONCLUSIVE);
19 return TimingResult(verdict = ERROR);
// a missing pair is not a point observation

```

Algorithm S6: COMMITCOMPATIBLEUPDATE: history-compatible update and summary commit (ALG-CLTAV-07, S9)

Require: SelectSnapshot ξ ; normalized Outcome z from the resolving module; Γ ; η ; H ; precondition enforced by the caller
 ALG-CLTAV-01: the S9 guard holds (a valid observation class or a confirmed successor summary)

Ensure: a tagged UpdateResult ($status, \eta', H', \Gamma'$) with status OK or SPEC-ERROR; Γ' initialized from Γ

```

1   $\Gamma' \leftarrow \Gamma$ ;
   //  $\Gamma'$  is a copy of  $\Gamma$ ; no field is left undefined
   // a stale or inconsistent history version fails closed before any update
2  if  $\xi.historyVersion \neq \eta.version$  then return
   UpdateResult(status = SPEC-ERROR,  $\eta, H, \Gamma'$ , reason = history version mismatch);
3  if  $z.I_z$  is a valid observation class then
4     $(\eta_c, H_c) \leftarrow$ 
       IF-HIST-UPDATE( $\eta, H, \xi.actionId, \xi.qUsedAtSelect, \xi.classesUsedAtSelect, \xi.historyVersion, z.I_z, z.postSummary$ );
       // IF-HIST-UPDATE is the sole history writer; it must agree with the intersection and the snapshot version
5    if  $H_c \neq H \cap z.I_z$  then return UpdateResult(status = SPEC-ERROR,  $\eta, H, \Gamma'$ , reason = intersection mismatch);
6     $\eta' \leftarrow \eta_c$ ;  $H' \leftarrow H_c$ ;
7  else
8     $(\eta', H') \leftarrow$ 
       IF-HIST-UPDATE( $\eta, H, \xi.actionId, \xi.qUsedAtSelect, \xi.classesUsedAtSelect, \xi.historyVersion, NONE, z.postSummary$ );
        $H' \leftarrow H$ ;
       // a confirmed successor summary with no valid observation advances history but preserves  $H$ 
9  if  $\Gamma'.qStatus$  is KNOWN and  $z.summaryConfirmed$  is true and  $z.postSummary$  is confirmed then
10    $\Gamma'.currentSummary \leftarrow z.postSummary$ ;
   // this is the sole commit of the confirmed successor summary
11 return UpdateResult(status = OK,  $\eta', H', \Gamma'$ );
```
